AeroSafe

Application intelligente de surveillance et d'analyse des données aéronautiques

---

Cours : Génie Logiciel et Développement Opérationnel — IPSA Bachelor 3

Groupe : Lucas, Teddy, Tov, Ryan

Version : V3 — 9 novembre 2026

Plan

1- Présentation générale

- Présentation du projet
- Contexte pédagogique
- Problématique
- Historique du document

2- Périmètre du projet

- Ce qui est inclus
- Ce qui est exclu

3- Spécification détaillée

- Fonctionnalités par profil
- Règles métier
- Alertes
- Écrans et maquettes

4- Expression des besoins

- Besoins fonctionnels
- Besoins non fonctionnels
- Cible / utilisateurs finaux

5- Conception

- Architecture
- Modèle de données
- Diagrammes UML
- Choix techniques

6- Méthode de travail

- Rôles
- Sprints
- Rituels et traces
- Outils et règles

7- Contraintes

- Contraintes techniques
- Contraintes budgétaires
- Contraintes de délai
- Contraintes réglementaires

8- Livrables attendus

- Liste des livrables
- Format et modalités de livraison

9- Aspect contractuel

- Parties
- Propriétés intellectuelles
- Confidentialité
- Pénalités

10- Réception et validation du projet

- Critères d'acceptation
- Modalités de test
- Critères d'évaluation et preuves

Présentation générale

1. Présentation du projet

AeroSafe est une application web qui récupère en continu les données de vol publiées par OpenSky Network, les enregistre, les affiche sur une carte et lève une alerte quand un avion a un comportement inhabituel. Elle sert deux profils : un utilisateur qui consulte un avion, et un superviseur qui surveille l'ensemble du trafic et ses alertes.

Le projet couvre tout le cycle de vie d'un logiciel : analyse des besoins, conception, développement, tests, intégration continue et déploiement.

2. Contexte pédagogique

AeroSafe est le projet fil rouge du cours de Génie Logiciel et Développement Opérationnel (IPSA, Bachelor 3). Il est réalisé par le groupe Lucas, Teddy, Tov et Ryan, dans une démarche proche d'une relation client et prestataire : l'IPSA exprime le besoin, l'équipe le spécifie, le réalise et le livre.

3. Problématique

Comment exploiter en quasi temps réel les données de vol d'OpenSky Network pour détecter automatiquement des situations anormales (perte de contact, position obsolète, variation brutale d'altitude, de vitesse ou de cap) et en informer un superviseur de façon fiable et compréhensible ?

4. Historique du document

Version	Auteurs	Date	Apport principal
V1	Teddy	14 sept. 2026	Première expression du besoin, structurée par chapitres
V2	Ryan, Lucas	15 sept. 2026	Plan du cours, périmètre, seuils des règles, écrans, diagramme de cas d'utilisation
V3	Lucas	9 nov. 2026	Version consolidée : seuils corrigés, diagrammes UML et maquettes du sprint 3, choix techniques, méthode Scrum, aspects contractuels

Toutes les versions sont conservées dans le dépôt, dossier `docs/cahier-des-charges/`.

Périmètre du projet

1. Ce qui est inclus

- **Collecte des données** : interrogation régulière de l'API OpenSky sur une zone définie et enregistrement de chaque observation dans PostgreSQL.
- **Consultation d'un aéronef** : recherche par identifiant ICAO24 ou par indicatif de vol, affichage de ses données de vol.
- **Analyse des données de vol** : position, altitude, vitesse, cap, évolution dans le temps et statistiques de vol.
- **Trajectoires** : reconstruction à partir de l'historique et affichage sur une carte interactive.
- **Moteur de règles** : application automatique des règles R1 à R5 (voir Règles métier).
- **Alertes** : création et enregistrement en base de chaque alerte, avec son explication.
- **Dashboard de supervision** réservé au superviseur.
- **Authentification et rôles** : profils Utilisateur et Superviseur.
- **Cycle de vie logiciel** : conception UML, tests automatisés, Git et GitHub, intégration continue, conteneurisation Docker, documentation.
- **Options**, si le temps le permet : niveau de gravité des alertes, module d'intelligence artificielle produisant un score d'anomalie comparé aux règles.

2. Ce qui est exclu

- **Usage opérationnel réel** : AeroSafe est un outil pédagogique. Il ne remplace aucun système de sécurité aérienne ni de contrôle du trafic.
- **Collecte en propre** : pas d'antenne ADS-B ; seules les données d'OpenSky Network sont utilisées.
- **Couverture mondiale** : la surveillance se limite à la France métropolitaine (latitude 41,3° à 51,1° N, longitude 5,2° O à 9,6° E), pour rester dans les quotas de l'API.
- **Application mobile native** : l'interface est web uniquement.
- **Notifications externes** : pas d'e-mail ni de SMS ; les alertes se consultent dans l'application.
- **Prédiction de trajectoire** : l'application analyse le passé et le présent, pas le futur.

Spécification détaillée

1. Fonctionnalités par profil

Le superviseur dispose de tout ce que fait l'utilisateur, plus la vue d'ensemble et la gestion des alertes et des règles.

Fonctionnalité	Utilisateur	Superviseur
Se connecter	oui	oui
Rechercher un aéronef (ICAO24 ou indicatif)	oui	oui
Consulter ses données de vol et leur évolution	oui	oui
Voir sa trajectoire sur une carte	oui	oui
Consulter les alertes d'un aéronef	oui	oui
Dashboard : aéronefs suivis, alertes actives, carte globale	non	oui
Filtrer et marquer une alerte comme traitée	non	oui
Activer, désactiver et régler les seuils des règles	non	oui

2. Règles métier

Les règles s'appliquent automatiquement à chaque nouvelle collecte. Les seuils sont exprimés dans les unités fournies par OpenSky (secondes, mètres par seconde, degrés) ; les équivalents aéronautiques sont donnés pour repère.

Règle	Ce qu'elle détecte	Donnée OpenSky	Seuil de départ
R1	Perte de contact : plus aucun message reçu	`last_contact`	plus de 5 min (300 s)
R2	Position trop ancienne : messages reçus, mais position non rafraîchie	`time_position`	plus de 10 min (600 s)
R3	Variation brutale d'altitude	`vertical_rate`	plus de 15 m/s en valeur absolue (environ 3 000 ft/min)
R4	Variation importante de vitesse entre deux observations	`velocity`	plus de 51 m/s (environ 100 kt) en moins de 60 s
R5	Changement brusque de cap entre deux observations	`true_track`	plus de 45° en moins de 60 s

Trois précisions d'implémentation :

- **R1 et R2 sont distinctes** : OpenSky donne séparément l'heure du dernier message et celle de la dernière position. R2 se déclenche quand l'avion reste visible sans être localisable.
- **R3 à R5 comparent deux observations successives** du même avion : l'historique complet doit donc être conservé.
- **R5 tient compte du cercle** : passer de 350° à 10° représente 20° d'écart, et non 340°.

La V2 indiquait pour R3 un seuil de 300 ft/s (18 000 ft/min), irréaliste : il est corrigé ici. Ces seuils sont des valeurs de départ, ajustées après quelques jours de collecte selon le nombre d'alertes produites, et modifiables par le superviseur.

3. Alertes

Chaque déclenchement d'une règle crée une alerte enregistrée en base, qui n'est jamais supprimée, seulement marquée comme traitée. Une alerte contient :

- l'aéronef concerné et la règle déclenchée ;
- la date et l'heure de détection (UTC) ;
- la valeur observée et le seuil en vigueur au moment du déclenchement ;
- un message explicatif lisible et son statut (en cours ou traitée) ;
- en option, un niveau de gravité.

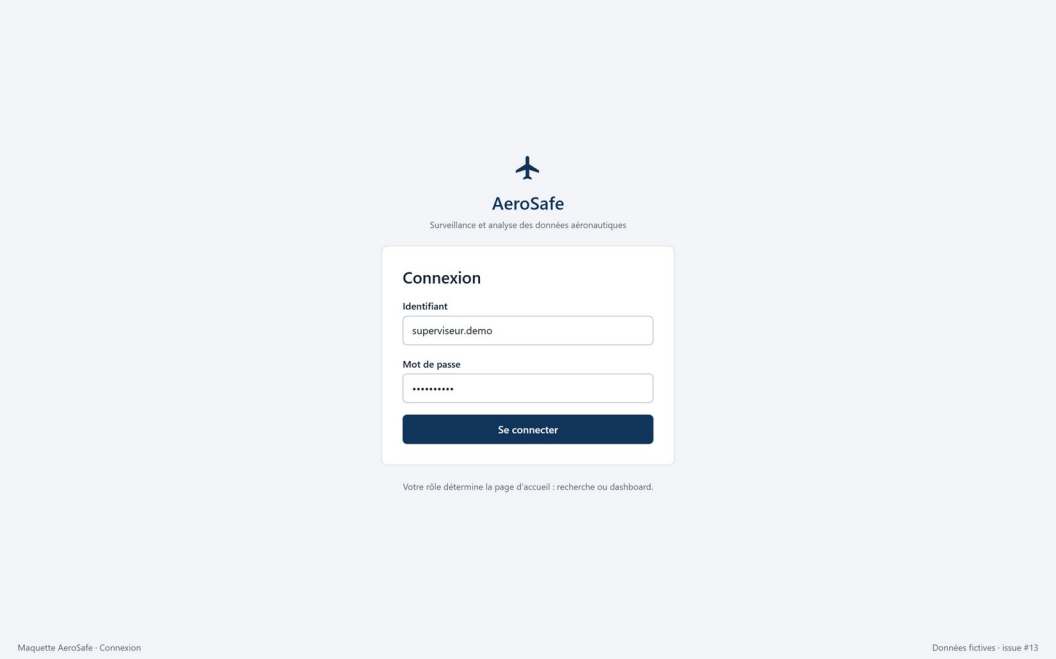
Une même situation ne produit qu'une seule alerte tant qu'elle dure.

4. Écrans et maquettes

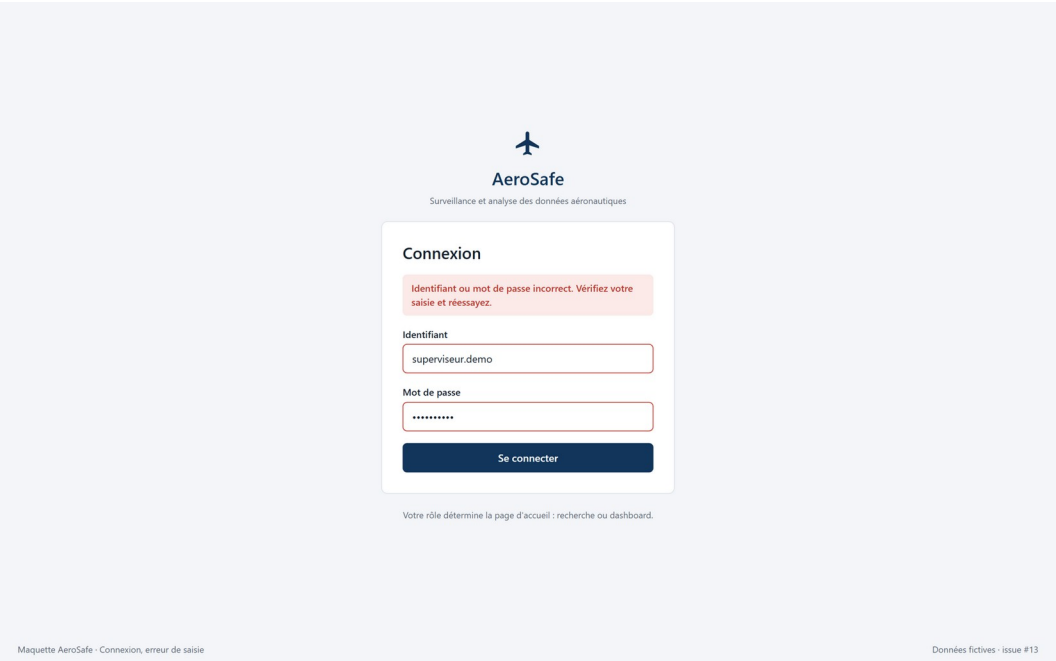
Écran	Profil	Contenu
Connexion	tous	Formulaire centré (identifiant, mot de passe), redirection selon le rôle
Recherche	tous	Barre de recherche en haut ; résultats avec identifiant, indicatif et statut
Fiche aéronef	tous	Valeurs et courbes d'altitude, de vitesse et de cap ; carte avec la trajectoire et les positions successives ; alertes de l'aéronef
Dashboard	superviseur	Bandeau (aéronefs suivis, alertes actives), tableau des alertes avec accès au détail, carte de tous les aéronefs

Les maquettes ci-dessous ont été réalisées au sprint 3 par Teddy ; leurs sources HTML sont dans `docs/conception/maquettes/`. Les données affichées sont fictives mais cohérentes d'un écran à l'autre : l'alerte R3 d'AFR1462 est la même sur le dashboard, dans la recherche et sur la fiche. Après la connexion, l'utilisateur arrive sur la recherche et le superviseur sur le dashboard ; le menu « Règles » n'est visible que du superviseur.

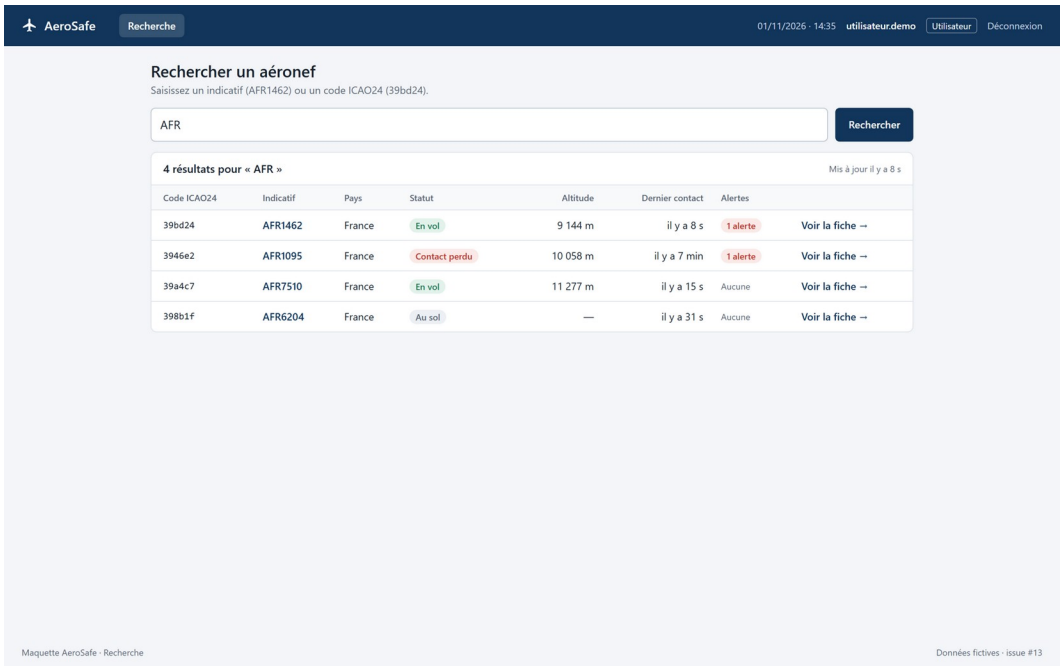
Connexion



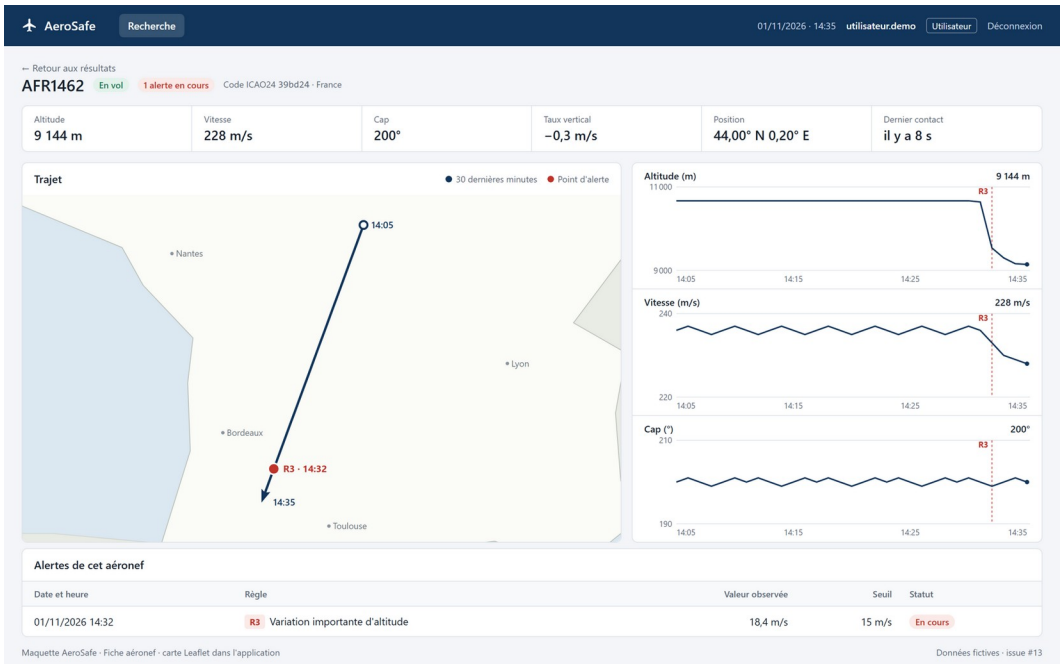
Connexion — erreur de saisie



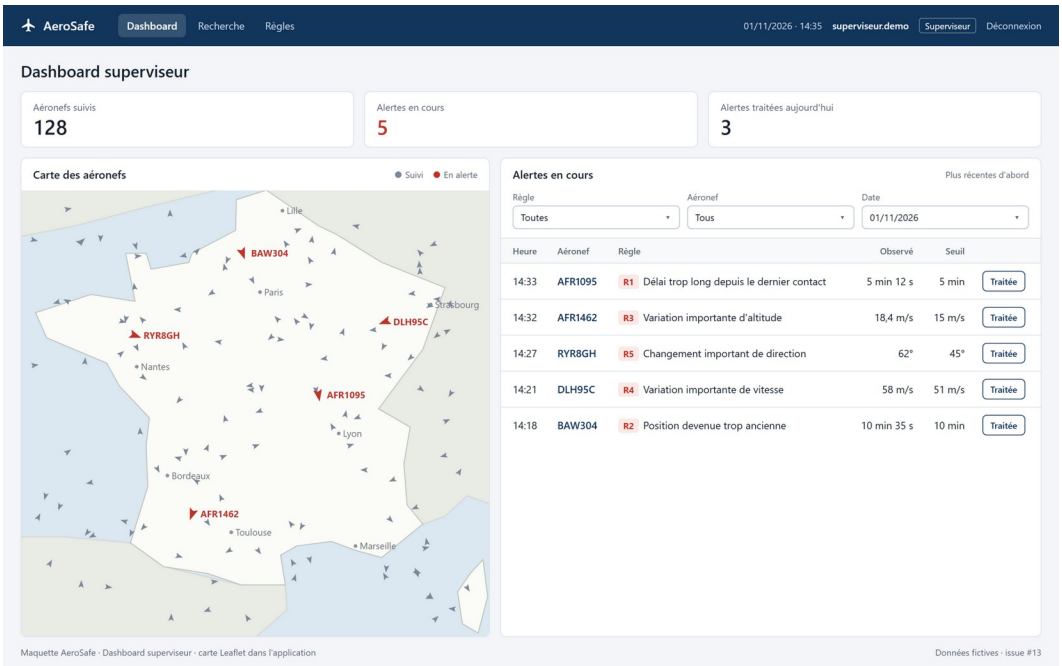
Recherche d'un aéronef



Fiche aéronef



Dashboard superviseur



Expression des besoins

1. Besoins fonctionnels

Chaque besoin correspond à une ou plusieurs tâches du backlog GitHub. La priorité suit la méthode MoSCoW : Must = indispensable, Should = attendu mais reportable, Could = bonus.

N°	Le système doit…	Priorité
BF-01	collecter les données OpenSky de la zone toutes les 60 secondes et conserver tout l'historique	Must
BF-02	permettre de rechercher un aéronef par ICAO24 ou par indicatif	Must
BF-03	afficher les données de vol d'un aéronef et leur évolution dans le temps	Must
BF-04	reconstruire la trajectoire d'un aéronef et l'afficher sur une carte	Must
BF-05	appliquer automatiquement les règles R1 à R5 et enregistrer une alerte à chaque détection	Must
BF-06	authentifier chaque personne et limiter l'accès selon son rôle	Must
BF-07	offrir au superviseur un dashboard des aéronefs suivis et des alertes actives	Must
BF-08	permettre au superviseur de filtrer les alertes et de les marquer comme traitées	Should

N°	Le système doit…	Priorité
BF-09	permettre au superviseur de modifier les seuils et d'activer ou désactiver une règle	Should
BF-10	attribuer un niveau de gravité à chaque alerte	Could
BF-11	calculer un score d'anomalie par apprentissage automatique et le comparer aux règles	Could

2. Besoins non fonctionnels

N°	Domaine	Exigence
BNF-01	Performance	Une recherche ou l'affichage d'une trajectoire répond en moins de 2 secondes sur le jeu de données de démonstration.
BNF-02	Fiabilité	Une panne ou un refus de l'API OpenSky n'arrête pas l'application : l'erreur est journalisée et la collecte reprend au cycle suivant.
BNF-03	Sécurité	Toutes les routes de l'API, sauf la connexion, exigent un jeton valide ; les mots de passe sont stockés hachés, jamais en clair.
BNF-04	Confidentialité	Aucun identifiant ni mot de passe dans le dépôt : ils sont fournis par un fichier .env non versionné.
BNF-05	Ergonomie	Un utilisateur non technique trouve un avion et comprend une alerte sans aide ; chaque alerte affiche sa raison.
BNF-06	Maintenabilité	Code Python conforme à PEP 8, organisé en classes, documenté par des docstrings et couvert par des tests automatisés.
BNF-07	Évolutivité	Ajouter une règle de détection revient à ajouter une classe, sans modifier le moteur.
BNF-08	Portabilité	L'application démarre entièrement avec une seule commande (docker compose up) sur tout poste équipé de Docker.
BNF-09	Traçabilité	Toute alerte est horodatée en UTC et conservée.

3. Cible / utilisateurs finaux

- **Utilisateur** : une personne qui veut suivre un vol précis, voir sa trajectoire et comprendre une alerte. Elle consulte, elle ne configure rien.
- **Superviseur** : une personne chargée de surveiller l'ensemble du trafic suivi et de réagir aux alertes. Elle a besoin d'une vue d'ensemble et de régler la sensibilité des règles.

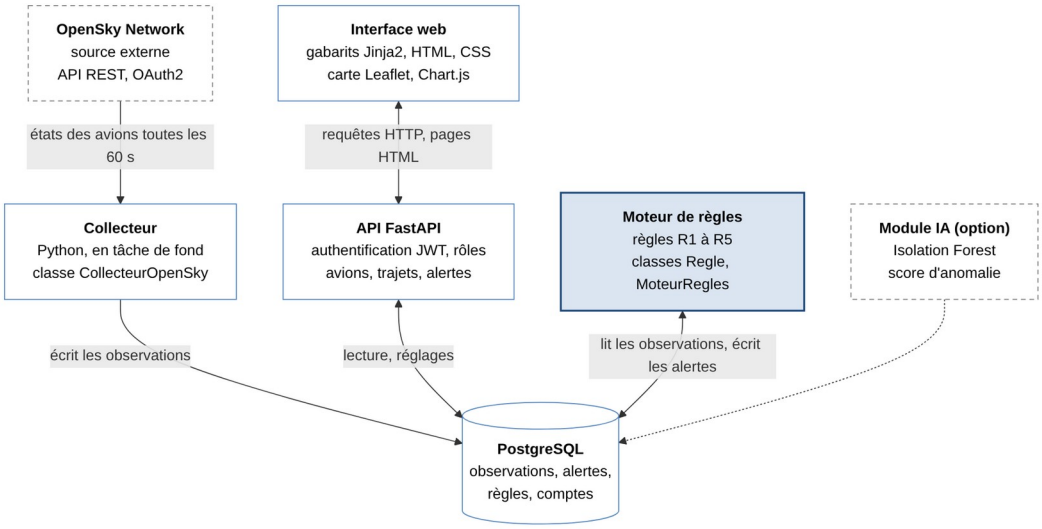
Pendant le projet, ces deux profils sont tenus par les membres du groupe, puis par le jury lors de la démonstration.

Conception

1. Architecture

L'application suit une architecture en trois couches : une interface web sans logique métier, une API FastAPI qui centralise les traitements, et une base PostgreSQL unique. La collecte et le moteur de règles tournent en tâche de fond, pour que la détection fonctionne même quand personne n'est connecté.

Architecture — AeroSafe



Le collecteur et le moteur de règles travaillent seuls sur la base ; l'interface ne passe que par l'API. Le module IA, optionnel, lit l'historique stocké dans PostgreSQL.

2. Modèle de données

Une observation n'est jamais écrasée : chaque collecte ajoute des lignes, ce qui permet de reconstruire une trajectoire et de comparer deux instants.

Table	Colonnes principales	Rôle
aircraft	icao24 (clé), callsign, origin_country, first_seen, last_seen	Un aéronef, identifié par son adresse ICAO24, stable contrairement à l'indicatif
observation	id, icao24, observed_at, latitude, longitude, baro_altitude, velocity, true_track, vertical_rate, on_ground, last_contact, time_position	Une position reçue à un instant ; index sur (icao24, observed_at)

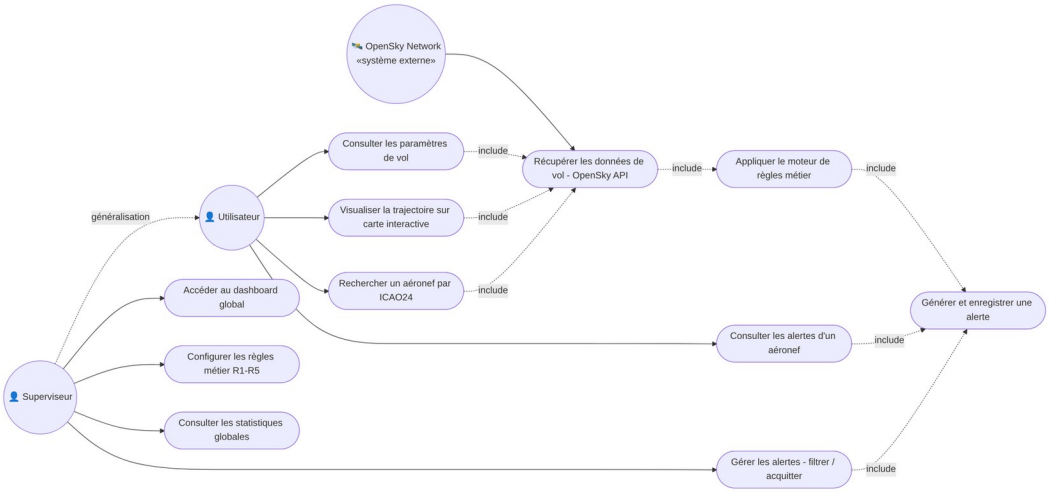
Table	Colonnes principales	Rôle
rule	code (R1 à R5), name, description, threshold, enabled	Les règles et leurs seuils, modifiables sans redéploiement
alert	id, icao24, rule_code, detected_at, observed_value, threshold, message, status, severity	Une alerte ; elle garde le seuil en vigueur au moment du déclenchement
app_user	id, username, password_hash, role	Un compte (rôle user ou supervisor) ; user est un mot réservé de PostgreSQL

Unités stockées : mètres, mètres par seconde, degrés, horodatages en UTC.

3. Diagrammes UML

Diagramme	Ce qu'il montre	Où le trouver
Cas d'utilisation	Les acteurs (Utilisateur, Superviseur, OpenSky) et leurs actions	docs/conception/ diagramme_cas_utilisation.md (sprint 3, Ryan)
Classes	Les classes du collecteur, du moteur de règles et des alertes	docs/conception/classes.md (sprint 3, Ryan)
Séquence	Le parcours d'une donnée : collecte → règles → alerte	docs/conception/sequence.md (sprint 3, Ryan)

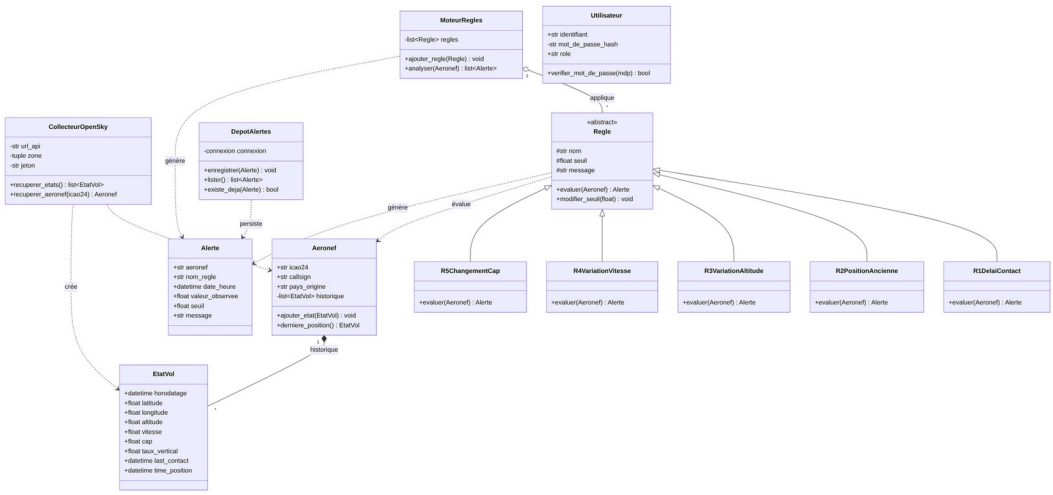
Diagramme de cas d'utilisation — AeroSafe



Le diagramme de classes guide la programmation orientée objet. Il s'organise en quatre groupes :

- **Collecte** : `CollecteurOpenSky` interroge l'API et crée les objets `Aeronef` et `EtatVol` ; un aéronef garde l'historique de ses états, nécessaire à R3, R4 et R5.
- **Règles** : la classe abstraite `Regle` définit `evaluer()`, que chaque sous-classe `R1DelaiContact` à `R5ChangementCap` écrit à sa façon. `MoteurRegles` applique la liste des règles sans connaître leur détail : ajouter une règle revient à ajouter une classe (BNF-07).
- **Alertes** : `Alerte` porte la valeur observée et le seuil ; `DepotAlertes` isole l'accès à PostgreSQL et évite les doublons avec `existe_deja()`.
- **Comptes** : `Utilisateur` porte le rôle et vérifie le mot de passe sans jamais l'exposer.

Diagramme de classes — AeroSafe



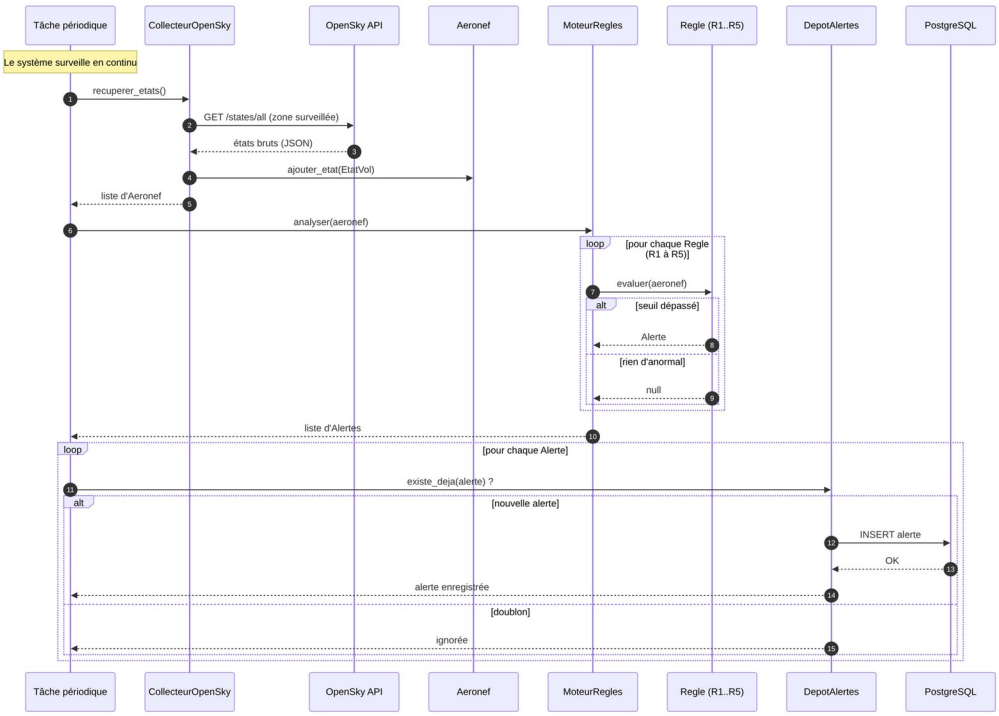
L'équipe a fixé une règle d'encapsulation : les données internes sont privées et ne se manipulent que par des méthodes publiques ; les attributs dont les sous-classes ont besoin sont protégés. En Python, ces membres s'écrivent avec un _ devant (par exemple `self._regles`).

Classe	Membres privés ou protégés	Accessibles par
`CollecteurOpenSky`	url_api, zone, jeton	recuperer_etats(), recuperer_aeronef()
`Aeronef`	historique	ajouter_etat(), derniere_position()
`Regle`	nom, seuil, message (protégés)	evaluer(), modifier_seuil()
`MoteurRegles`	regles	ajouter_regle(), analyser()
`DepotAlertes`	connexion	enregistrer(), lister(), existe_deja()
`Utilisateur`	mot_de_passe_hash	verifier_mot_de_passe()

`EtatVol` et `Alerte` restent publics : ce sont de simples conteneurs de données. Les classes correspondent aux tables du modèle de données : `Aeronef` → aircraft, `EtatVol` → observation, `Alerte` → alert, `Utilisateur` → app_user.

Le diagramme de séquence montre l'ordre des appels pour une collecte : la tâche périodique appelle `recuperer_etats()`, puis `analyser()` sur chaque aéronef ; le moteur appelle `evaluer()` sur chaque règle, et chaque alerte nouvelle est enregistrée après vérification par `existe_deja()`.

Diagramme de séquence — AeroSafe



4. Choix techniques

Couche	Choix	Raison
Langage	Python 3.11	Langage commun à l'équipe, imposé par FastAPI
API	FastAPI	Imposée par le sujet ; documentation de l'API générée automatiquement
Base de données	PostgreSQL	Imposée par le sujet ; robuste sur de gros volumes
Accès aux données	SQLAlchemy	Manipuler la base avec des classes Python plutôt qu'avec du SQL dispersé

Couche	Choix	Raison
Interface	Gabarits Jinja2 rendus par FastAPI, HTML et CSS des maquettes	Les pages sont générées en Python ; seuls la carte et les graphiques demandent un peu de JavaScript (Leaflet, Chart.js)
Carte	Leaflet	Bibliothèque de cartes libre, légère et bien documentée
Authentification	Jetons JWT, mots de passe hachés (bcrypt)	Adapté à une API, simple à tester
Tests et qualité	pytest, ruff	Tests unitaires et d'intégration, vérification du style
Conteneurs	Docker, Docker Compose	Même environnement pour les quatre membres, démonstration en une commande
Intégration continue	GitHub Actions	Tests lancés automatiquement à chaque pull request
IA (option)	scikit-learn, Isolation Forest	Détection d'anomalies sans données étiquetées

Ces choix ont été confirmés en équipe pendant le sprint 3. La structure du dépôt suit cette architecture : backend/, collector/, frontend/ et tests/, avec PostgreSQL lancé par Docker Compose.

Méthode de travail

L'équipe applique Scrum depuis le début du projet, et tout le travail est suivi sur GitHub : github.com/aerosafe-ipsa/aerosafe.

1. Rôles

Membre	Rôle Scrum	Référent technique
Lucas	Product Owner : tient et priorise le backlog, valide les tâches terminées	Données et base
Tov	Scrum Master : anime les rituels, fait respecter la méthode, administre le dépôt	Qualité et DevOps
Ryan	Développeur	Moteur de règles et API
Teddy	Développeur	Interface web

2. Sprints

Les sprints durent deux semaines pendant le cadrage, puis une semaine à partir du sprint 3 pour multiplier les points de contrôle avant le rendu.

Sprint	Dates	Contenu
S0	14 → 27 sept.	Cadrage, cahiers des charges V1 et V2
S1	28 sept. → 11 oct.	Étude de l'API OpenSky
S2	12 → 25 oct.	Premier dépôt et test de l'API
S3	26 oct. → 1er nov.	Scrum sur GitHub, conception UML, maquettes, cahier des charges V3
S4	2 → 8 nov.	Collecte, base de données, connexion
S5	9 → 15 nov.	Règles et API
S6	16 → 22 nov.	Interface, intégration continue
S7	23 → 29 nov.	Finitions, version v1.0
S8	30 nov. → 6 déc.	Rendu du 1er décembre, préparation de la soutenance du 9 décembre

Le rendu tombe pendant le sprint 8 : la version v1.0 est livrée à la fin du sprint 7, et la dernière semaine sert à préparer la soutenance.

3. Rituels et traces

Rituel	Quand	Trace sur GitHub
Planning	Lundi	Tâches placées dans le milestone du sprint, compte rendu dans docs/scrum/sprints/
Daily	Du lundi au vendredi, à l'écrit	Discussion « Daily – Sprint X »
Revue	Dimanche	Tâches acceptées ou refusées par le Product Owner
Rétrospective	Après la revue	Ce qui marche, ce qui bloque, une action pour le sprint suivant

4. Outils et règles

- **Backlog** : chaque tâche est une issue GitHub, avec un objectif, des critères de validation, une priorité et une estimation en points (1, 2, 3, 5 ou 8).
- **Tableau** : GitHub Projects, colonnes Backlog, Sprint, En cours, En revue, Terminé.
- **Code** : la branche main est protégée ; chaque tâche passe par une branche et une pull request relue par un autre membre.

- **Versions** : un tag et une release GitHub en fin de sprint, décrits dans CHANGELOG.md.
- **Définition du « terminé »** : code fusionné par pull request relue, tests qui passent, docstrings présentes, critères de validation vérifiés par le Product Owner.

Contraintes

1. Contraintes techniques

- Architecture imposée par le sujet : interface web, API FastAPI, base PostgreSQL.
- Source de données unique : l'API d'OpenSky Network, avec authentification OAuth2 et un quota de crédits journalier. Un refus pour quota dépassé (code 429) doit être géré sans arrêter l'application.
- Les données ADS-B sont souvent incomplètes (position, altitude ou cap absents) : chaque traitement doit accepter les valeurs manquantes.
- La démonstration se fait en direct : un jeu de données enregistré à l'avance sert de secours si l'API ne répond pas.

2. Contraintes budgétaires

Le projet n'a aucun budget : seuls des outils gratuits ou libres sont utilisés (organisation et dépôt GitHub publics, GitHub Actions, Docker, PostgreSQL, compte OpenSky gratuit). La charge de travail représente environ 12 semaines pour quatre étudiants, en parallèle des autres cours.

3. Contraintes de délai

Échéance	Date
Début du projet	14 septembre 2026
Envoi du cahier des charges, du lien GitHub et ajout de l'enseignante comme collaboratrice	1er décembre 2026
Soutenance	9 décembre 2026 à 14 h

La date de fin ne bouge pas : en cas de retard, ce sont les tâches Could, puis Should, qui sont abandonnées.

4. Contraintes réglementaires

- **OpenSky Network** : usage non commercial uniquement, conformément à ses conditions d'utilisation ; la source des données est citée dans l'application et dans la documentation.
- **RGPD** : seules les données nécessaires aux comptes sont conservées (identifiant, mot de passe haché, rôle). Les comptes de démonstration ne contiennent aucune donnée personnelle réelle.

- **Aéronefs** : l'application n'affiche que des données publiques diffusées par les avions eux-mêmes.

Livrables attendus

1. Liste des livrables

Livrable	Contenu	Forme	Échéance
Cahier des charges et conception	Ce document : besoins, règles, conception UML, architecture, modèle de données, choix techniques, méthode	PDF	1er décembre 2026
Code source	Interface web, API FastAPI, collecteur, moteur de règles, scripts de base de données, tests, fichiers Docker	Dépôt GitHub public	1er décembre 2026
Documentation	README, guide de contribution, documentation générée depuis les docstrings, comptes rendus de sprint	Dans le dépôt	1er décembre 2026
Démonstration	Application lancée en direct, sans vidéo préenregistrée	10 minutes	9 décembre 2026
Présentation	Projet, méthode Scrum, organisation, conception, choix techniques, versions successives	PowerPoint (.pptx), 20 minutes	9 décembre 2026

2. Format et modalités de livraison

Le 1er décembre au plus tard, un seul mail pour le groupe est envoyé à Mme Sboui. Il contient le nom du groupe et de ses membres, ce cahier des charges, et le lien du dépôt GitHub. L'adresse sirine.sboui@ipsa.fr est ajoutée comme collaboratrice du dépôt.

La version livrée correspond à la release v1.0.0 du dépôt.

Aspect contractuel

1. Parties

Rôle	Qui
Client (maîtrise d'ouvrage)	L'IPSA, représentée par Mme Sirine Sboui, enseignante du cours de Génie Logiciel et Développement Opérationnel
Prestataire (maîtrise d'œuvre)	Le groupe Lucas, Teddy, Tov et Ryan

Le client fournit le sujet (fiche projet du 9 septembre 2026), fixe les échéances et évalue le résultat. Le prestataire spécifie, réalise, teste et livre l'application.

2. Propriétés intellectuelles

- Le code et la documentation sont produits par les membres du groupe, qui en restent les auteurs. Le client peut les utiliser librement pour l'évaluation et l'enseignement.
- Le dépôt est public pour permettre l'évaluation. Aucune licence libre n'y est attachée : le code peut être consulté, mais pas réutilisé sans l'accord de ses auteurs.
- Les données de vol restent la propriété d'OpenSky Network et ne sont utilisées que dans le cadre non commercial prévu par ses conditions.
- Les bibliothèques utilisées sont libres et gardent leurs propres licences.

3. Confidentialité

- Le code étant public, aucune information sensible n'y est déposée : les identifiants OpenSky et les secrets de l'application sont fournis par un fichier .env non versionné.
- Les comptes de démonstration n'utilisent ni données personnelles réelles, ni mots de passe réutilisés ailleurs.
- Si un secret est publié par erreur, il est immédiatement révoqué et remplacé.

4. Pénalités

Aucune pénalité chiffrée n'est prévue par le client. Le respect de l'échéance du 1er décembre 2026 est en revanche demandé explicitement : un livrable envoyé en retard peut être pénalisé dans l'évaluation, selon les règles de l'IPSA.

Réception et validation du projet

1. Critères d'acceptation

Le projet est conforme si :

- tous les besoins Must (BF-01 à BF-07) fonctionnent et leurs tâches sont fermées sur GitHub ;
- l'application démarre avec docker compose up sur un poste neuf ;
- chaque règle R1 à R5 produit une alerte sur un cas de test qui dépasse son seuil, et aucune sur un cas normal ;
- les tests automatisés passent dans l'intégration continue sur la version v1.0.0 ;
- la démonstration en direct déroule le scénario prévu devant le jury.

2. Modalités de test

Niveau	Ce qui est testé	Comment	Quand
Tests unitaires	Chaque règle (cas normal, seuil dépassé, valeur manquante, passage 350° → 10° pour R5), la lecture des données OpenSky	pytest	À chaque pull request
Tests d'intégration	Les routes de l'API et l'accès à la base, dont les droits selon le rôle	pytest	À chaque pull request
Recette de sprint	Les critères de validation de chaque tâche	Vérifiés par le Product Owner	En revue de sprint
Recette finale	Le scénario complet de démonstration	Répété deux fois, puis présenté au jury	Du 30 novembre au 9 décembre

3. Critères d'évaluation et preuves

Critère évalué	Où le voir
Cahier des charges	Ce document et ses versions dans docs/cahier-des-charges/
Conception	Diagrammes UML, architecture et modèle de données (chapitre Conception, docs/conception/)
Git et GitHub	Branches, pull requests relues, releases, protection de main
Méthode Scrum	Tableau Projects, milestones par sprint, comptes rendus, dailies
Qualité du code	Intégration continue, tests, style vérifié par ruff
Programmation orientée objet	Moteur de règles, collecteur et modèles organisés en classes
Documentation et docstrings	README, docstrings, documentation générée
Démonstration en direct	Scénario minuté dans docs/demo.md
Présentation orale	Support PowerPoint de 20 minutes